Experiment 2 — Machine arrays and fused elementwise loops

Mathilda — execution-speed experiments

Contents

Experiment 2 — Machine arrays and fused elementwise loops

Date: 2026-07-27 · Commits: `b6499fa` (M3a), `a4b6cdf`, `3329332` (M5b), `49629fe`, `cbbdc58` (M3c) · Code: `src/compile/` · Result: fusion 2.4–3.4×, threading 3.2–6.6×

Common method in `README.md`.

Hypothesis

Experiment 1 made scalar numeric code fast. Array code was still slow for a different reason: `Sqrt[v] + v^2` over a million elements allocates three million-element temporaries and makes three passes over memory, where one pass computing `sqrt(v[i]) + v[i]*v[i]` would do.

This is loop fusion, and it is the single largest structural win available to an array language — NumPy famously cannot do it (`np.sqrt(v) + v**2` materialises both temporaries), which is why `numexpr` and `numba` exist.

What was built

M3a — rank-1 machine arrays. An `_Real` array argument reaches the VM as a raw `double*` with no boxing at the boundary.

M3b/M5 — any rank, and opt-in elementwise fusion. An expression tree over array operands is lowered to a single loop over the flat index, with the intermediate values living in registers rather than in memory.

M5b — strip mining. The first fusion implementation processed one element at a time through the whole expression, which defeated vectorisation: the compiler could not see a loop it could widen. Fusion now runs in blocks of `VBLOCK = 64`, so each stage of the expression is a short vectorisable loop over a cache-resident block. This is what made fusion pay — it was roughly break-even before, and is now on by default.

Threading. The fused loop splits across cores. Elementwise array work is embarrassingly parallel and every kernel is a pure function.

M3c — indexed arrays. `Part` read and write inside a compiled body, which is what a stencil needs. Worked example: `COMPILE_EXAMPLE.md`.

Results

Fusion (M5b), ns/element at 10^6

Measured within Mathilda, fusion off vs on:

body	ns/element	speedup
`v^2 + 2 v + 1`	2.3	3.4×
`Total[v^2 + 2 v + 1]`	2.8	3.0×
`Total[v + v v]`	1.7	2.4×
`Total[Sin[v] Exp[−v] + Sqrt[v]]`	12.9	1.9× (libm-bound)

The last row is the shape of the result: fusion saves memory traffic, so its benefit is bounded by how much of the time is memory traffic. A body dominated by `Sin` and `Exp` is bounded by libm, and 1.9× is what remains.

Threading the fused loop, 10^6 elements

body	serial	threaded	speedup
`Sqrt[v] + v^2`	2524 µs	782 µs	3.2×
`Sin[v] Exp[−v] + Sqrt[v]`	14376 µs	2611 µs	5.5×
`Gamma[v] + Erf[v]`	35900 µs	5479 µs	6.6×

Speedup rises with the cost of the body, as it must: a memory-bound body is limited by one memory system however many cores read it.

Against Mathematica and NumPy

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Sin (elementwise)	13.53 ms	11.71 ms	52.68 ms	1/1.16x	3.89x
Exp (elementwise)	13.37 ms	11.50 ms	54.86 ms	1/1.16x	4.10x
STREAM triad, a = b + 3 c	33.30 ms	31.82 ms	29.62 ms	1/1.05x	1/1.12x

The interesting comparison is the fused one, because it is the thing NumPy cannot do:

body, 10^6 elements	Mathilda `Compile[]` (fused)	NumPy (unfused)
`v^2 + 2 v + 1`	2.3 ms	3.56 ms
`Sqrt[v] + v^2`	0.78 ms (threaded)	2.95 ms
`Sin[v] Exp[−v] + Sqrt[v]`	2.61 ms (threaded)	16.6 ms

NumPy materialises every intermediate — `np.sqrt(v) + v**2` allocates two million-element temporaries and makes three passes. Mathilda's fused loop makes one. This is the one structural advantage a compiler has over an array library, and it is why `numexpr` and `numba` exist.

The measurement trap that nearly buried this

The first threading measurements came back at 0.56–0.83× and looked like a clear regression. They were not. Two separate mistakes, both worth recording:

1. **`clock()`** measures CPU time, not wall time. On a threaded path CPU time rises by roughly the thread

count even as wall time falls. Instrumenting the region directly showed it running 6.4× faster than the "regression" suggested. The same trap in CAS form is `Timing[]` vs `AbsoluteTiming` — which is why the method section of every one of these experiments says which is used.

2. The test build was **`-O0`**. M5's optimiser measured 1.53× on Horner at `-O0` and essentially nothing at `-O3`, because at `-O3` the C compiler was already doing what the bytecode optimiser had just been taught to do. The figures were re-measured and the changelog corrected (`cb3d978`). A compiler-optimisation benchmark run at **`-O0`** measures the wrong compiler.

Both are recorded in `tasks/lessons.md`.

Still open

- Fusion covers elementwise chains and reductions over them. It does not cover a scan (see `HPC_SWEEP_3_NUMPY_GAP.md`) or a gather.
- k-means and logistic regression are the two application kernels still ~3× behind NumPy, and both are compositions of unfused array operations outside `Compile[]`. Fusion at the interpreter level — not just inside `Compile[]` — is the natural next step, and is where the remaining factor of three is.

Why Mathilda is not the fastest here, and what it would take

Against NumPy the fused compiled path is ahead everywhere it was measured — 1.5× to 6.4×, because NumPy does not fuse at all. Against Mathematica three rows are marginally behind:

row	Mathilda	Mathematica	gap	cause
`Sin` elementwise, 10^7	13.53 ms	11.71 ms	1.16×	libm's `sin`, one element at a time
`Exp` elementwise, 10^7	13.37 ms	11.50 ms	1.16×	libm's `exp`, likewise
STREAM triad, 10^7	33.30 ms	31.82 ms	1.05×	at the memory floor in both

The triad row is not a gap worth chasing: 33.3 ms for 240 MB of traffic is 7.2 GB/s, both systems are within 5% of each other, and the machine's ceiling is the ceiling.

The two transcendental rows are a real 16%, and the cause is specific: Mathilda calls libm's scalar `sin`/`exp` per element, where a vector math library computes four or eight at a time.

The road to fastest

1. A vector math library for the transcendental kernels. macOS ships Accelerate's `vvsin`, `vvexp`, `vvlog`, `vvpow` and friends, which are already linked — `MATHILDA_USE_ACCELERATE` is on. Routing `ndarray_map_unary`'s float64 arm through `vv*` where one exists is a table of function pointers, not an algorithm. Expected: 13.5 ms → ~7 ms, i.e. ahead of Mathematica, and it applies to every elementwise transcendental in the system at once.
2. Keep the scalar kernel as the fallback, because `vv*` covers only the common functions and the compile VM needs a scalar entry point regardless.

This is the highest value-per-line item in the whole roadmap: one dispatch table, and every transcendental array operation in Mathilda gets faster.